

AI-Powered TBI Rehabilitation Platform — Project Report

AI-Powered Traumatic Brain Injury Rehabilitation Platform

A comprehensive guide to the goals, architecture, technologies, and setup of this AI-driven rehabilitation ecosystem — designed so you can quickly re-orient yourself any time you return to this project.

Project Vision & Goals

Traumatic Brain Injury (TBI) and stroke frequently leave patients with severe motor, speech, and cognitive deficits requiring repetitive, long-term, personalised rehabilitation. Traditional systems fall short in four critical ways:

Therapist Scarcity

High-frequency professional sessions are inaccessible for most patients due to cost and scheduling.

Subjective Tracking

Progress is measured by observation alone — no granular, objective clinical data.

No Personalisation

Static exercise sheets cannot adapt to a patient's daily performance, causing frustration or boredom.

Slow Diagnostics

Radiologists need fast, explainable tools to screen CT scans for intracranial hemorrhages.

The platform addresses each of these with an integrated AI-driven ecosystem:

High-Level Architecture

The platform is a modular three-tier application : one React frontend communicating with two independent, specialised Python backends — each with a distinct responsibility.

System Architecture Overview

Technology Stack

Every component and the key technology powering it:

CT Scan Analysis Backend

Located in `./backend/` — classifies head CT scans for Intracranial Hemorrhage (ICH) .

ML Model — `model.py` & `predict.py`

EfficientNet-B4 fine-tuned for multi-label hemorrhage detection

Classifier head: Dropout(0.4) → Linear(512) → BatchNorm → ReLU → Dropout(0.3) → Linear(5)

Decision thresholds per class: [0.45, 0.5, 0.5, 0.45, 0.45]

The 5 detected hemorrhage sub-types:

Epidural — Bleeding between the skull bone and the outer membrane (dura mater).

Intraparenchymal — Bleeding directly inside the brain tissue.

Intraventricular — Bleeding inside the brain's fluid-filled chambers (ventricles).

Subarachnoid — Bleeding in the space surrounding the brain, beneath the arachnoid membrane.

Subdural — Bleeding between the dura mater and the arachnoid membrane.

Explainability — Grad-CAM (`gradcam.py`)

Generates heatmap overlays so clinicians can see why the model flagged a region

Hook Activation — Forward hooks on `model.backbone.features[-1]` extract the final convolutional feature maps.

Hook Gradients — Backward hooks capture gradients for the predicted class during backpropagation.

Weight Calculation — Gradients are averaged spatially (global average pooling) to obtain importance weights per feature map.

Heatmap Generation — Weighted activation sum → ReLU filter → resize to 224×224 → `cv2.COLORMAP_JET` colour map applied.

Overlay — Blends greyscale CT scan with coloured heatmap. Red/yellow = high attention (suspect lesion), blue/green = low attention.

Speech Therapy Backend

Located in `./tbi_therapy (1)/tbi_therapy/` — orchestrates speech sessions, facial monitoring, RL planning, and result storage.

Speech Analysis Pipeline — `speech_analysis.py`

4-stage phoneme transcription and scoring pipeline

Why Allosaurus? Standard speech engines fail for TBI patients because of dysarthria (slurred/distorted speech). Allosaurus is an unsupervised multilingual model that extracts raw phonemes directly — bypassing vocabulary matching entirely.

Phoneme Folding ensures minor dialect differences (e.g. /aa/ vs /ao/ mapped to the same bucket) don't unfairly penalise patients. Score formula: $\text{Accuracy} = (\text{Correctly aligned phonemes} / \text{Total reference phonemes}) \times 100\%$

Reinforcement Learning Planner — `rl_planner.py`

PPO Actor-Critic agent that behaves like a live adaptive therapist

State Space (5 inputs)

- Severity level (Severe→Normal, normalised 0–1)
- Recent accuracy (avg phoneme accuracy, last few attempts)
- Accuracy trend (improvement vs decline vs previous session)
- Session count (total sessions practiced)
- Fatigue proxy (derived from pause ratio in patient speech)

Action Space & Rewards

- 0 — Easier : decrease difficulty 1 level
- 1 — Same : maintain current level
- 2 — Harder : increase difficulty 1 level

Reward signals: +reward for pronunciation improvement +1.0 for maintaining high accuracy at harder level –1.5 if accuracy drops below 40% after advancing –0.5 if patient exits early/gives up

Therapy Difficulty Levels:

■ RL model weights auto-save to `./models/ppo_actor.pt` and `./models/ppo_critic.pt` after every session — patient difficulty policies persist across program runs.

Facial Feature Analysis — `facial_analysis.py`

MediaPipe Face Landmarker monitoring during speech exercises

Lip Closure Score — Measures upper-to-lower inner lip distance. If lips stay open during sounds that require closure (e.g. /p/, /m/), it flags reduced motor strength.

Jaw Vertical Position — Compares chin coordinates to forehead baseline. Checks whether jaw range of motion is normal or constricted.

Facial Asymmetry — Calculates relative horizontal difference of left/right symmetry pairs (mouth corners, eye corners, cheeks) relative to facial midline. High asymmetry = facial droop from unilateral stroke/TBI.

Database Schema

A local SQLite3 database (`session_manager.py`) stores all data with indices for fast lookups. Three tables form a one-to-many hierarchy:

Setup & Run Instructions

Each of the three modules starts independently. Open three separate terminals.

Prerequisites: Python 3.9–3.11, Node.js, and ffmpeg installed and in system PATH (ffmpeg is required by Librosa to parse .webm audio from browsers).

CT Scan Backend

Open a terminal in the `./backend/` folder and create a virtual environment:

Install dependencies:

Verify `best_model.pth` exists inside `./backend/` — this file holds the trained EfficientNet-B4 weights. The server will fail to start without it.

Start the FastAPI server:

Swagger API docs available at: <http://127.0.0.1:8001/docs>

Speech Therapy Backend

Open a terminal in the `./tbi_therapy (1)/tbi_therapy/` folder and create a virtual environment:

Install dependencies:

Download the CMU Pronouncing Dictionary to the `./data/` directory:

Place the MediaPipe model file `face_landmarker.task` inside the `./models/` directory.

Initialise the SQLite database:

Start the Flask server:

Server starts at: <http://127.0.0.1:5000>

React Frontend

Open a terminal in the `./frontend/` folder and install packages:

Ensure the `.env` file is configured with the correct base URLs:

Start the development server:

Open <http://localhost:3000> — grant microphone and camera permissions when prompted.

Tips, Troubleshooting & Extensibility

Speech Prediction Failure

If speech analysis returns an error, verify ffmpeg is in your system PATH. It is required by Librosa to decode .webm audio uploaded from modern browsers.

Adding New Therapy Words

Edit the THERAPY_PROTOCOLS dictionaries in modules/severity.py . Word lists are dynamically picked based on the current active difficulty tier.

RL Model Persistence

Actor/critic weights are auto-saved to ./models/ppo_actor.pt and ./models/ppo_critic.pt after every session — patient policies persist across restarts.

CT Model Weights

The file best_model.pth in ./backend/ is required for the CT backend to start. Confirm it exists before launching Uvicorn.

Browser Permissions

The frontend requires both microphone and camera access. On first load, the browser will prompt for permissions — allow both for speech and facial analysis to work.

API Documentation

FastAPI auto-generates interactive Swagger docs at <http://127.0.0.1:8001/docs> — useful for testing CT scan endpoints directly without the frontend.